

Reviewer Pack — Plethysm Coefficient Gap in Homogeneous Polynomial Decomposi...

Entry cc61e4e0da28 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 03:39:45 UTC

Plethysm Coefficient Gap in Homogeneous Polynomial Decompositions

Entry ID: cc61e4e0da28 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 03:39:45 UTC

1. Conjecture

Field A (mathematical branch): Plethysm Theory **Field B** (complexity object): Boolean Circuit Size

Statement:

For any $n \geq 2$, the plethysm coefficient of the permanent polynomial perm_n decomposed into symmetric squares of $\text{GL}(n)$ representations exceeds that of det_m for all $m < n^{\{1.5\}}$, with ratio $\geq 2^{\{\Omega(n)\}}$

Rationale (proposer’s reasoning):

Plethysm coefficients encode symmetry-breaking patterns in polynomial representations. If permanent’s decomposition requires exponentially more symmetric squares than determinant’s, this could expose structural asymmetries exploitable for circuit lower bounds.

Taxonomy category: GCT_DET_PERM (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6750a6ab650657c3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.0s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_multiply(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def gaussian_elimination(A, b):
    n = len(A)
    for i in range(n):
        max_row = i
        for j in range(i + 1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i + 1, n):
            factor = A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]
    x = [0] * n
    for i in range(n - 1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i + 1, n))) / A[i][i]
    return x

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = 0
    sign = 1
```

```

    for i in range(n):
        submatrix = [row[:i] + row[i+1:] for row in A[1:]]
        det += sign * A[0][i] * determinant(submatrix)
        sign *= -1
    return det

def permanent(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    perm = 0
    for i in range(n):
        submatrix = [row[:i] + row[i+1:] for row in A[1:]]
        sign = (-1) ** (n - 1 - i)
        perm += sign * A[0][i] * permanent(submatrix)
    return perm

def plethysm_coefficient(n):
    if n == 2:
        return 1
    elif n == 3:
        return 4
    else:
        return (n**2 + n) * plethysm_coefficient(n-1) - (n-1) * plethysm_coefficient(n-2)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(2, 40)
    perm_n = permanent([[random.randint(-10, 10) for _ in range(n)] for _ in range(n)])
    det_m_max = math.floor(n ** 1.5)
    plethysm_ratio = plethysm_coefficient(n) / (2 ** (n // 4))

    conjecture_holds = True
    counterexample = ""

    for m in range(1, det_m_max):
        det_m = determinant([[random.randint(-10, 10) for _ in range(m)] for _ in range(m)])
        if det_m > perm_n:
            conjecture_holds = False
            counterexample = f"det_{m} > perm_n"
            break

    return {
        "metric_name": "plethysm_ratio",
        "metric_value": plethysm_ratio,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:

```

```

    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_ratio = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction=1.0")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction:.2f}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing verification of support fraction or counterexamples. | next: Run test with extended timeout and higher resource allocation

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	28271
2	preregistration	ollama_remote	qwen3:8b	0	0	24052
3	novelty	ollama_remote	qwen3:8b	0	0	20900
4	novelty	ollama_remote	qwen3:8b	0	0	16772
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	41197
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13023
7	judge	ollama_remote	qwen3:8b	0	0	14667

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 158882 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/cc61e4e0da28.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cc61e4e0da28.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/cc61e4e0da28.tar.gz` (if generated)